

实验2报告：基于 Vision Transformer 的 CIFAR-10 图像分类

1. 实验目的

本实验旨在通过实现和训练 Vision Transformer (ViT) 模型，完成 CIFAR-10 图像分类任务，并分析模型的训练过程与性能表现。具体目标包括：

- 理解 Vision Transformer (ViT) 在图像分类任务中的基本原理与实现方式。
- 掌握 CIFAR-10 数据集上的训练、验证、日志记录与可视化流程。
- 通过训练曲线分析模型收敛性，并评估 ViT 在中小规模数据集上的分类效果。

2. 实验环境

2.1 软件环境

- Python 3.x
- torch $\geq$ 2.0.0
- torchvision $\geq$ 0.15.0
- matplotlib $\geq$ 3.7.0

2.2 实验数据集 (CIFAR-10)

CIFAR-10 是一个常用的图像分类数据集，包含 60,000 张 32x32 彩色图像，分为 10 个类别，数据集划分如下：

- 训练集：50,000 张彩色图像
- 测试集：10,000 张彩色图像
- 分类类别：10 类（飞机plane、汽车car、鸟bird、猫cat、鹿deer、狗dog、青蛙frog、马horse、船ship、卡车truck），每个类各有5000张训练图像和1000张测试图像。

3. 实验原理

本实验使用 Vision Transformer (ViT) 模型，针对训练任务设计了以下核心组件：

- Patch Embedding**：将输入图像切分为固定大小的 patch，并通过线性变换映射为 token 序列。
- Transformer Encoder**：由多层 Transformer Encoder 组成，包含多头自注意力机制和前馈神经网络，用于提取图像的全局特征。
- 分类头**：在 Transformer Encoder 输出的基础上，使用一个线性层进行分类预测。模型训练过程中，使用交叉熵损失函数优化分类性能，并通过 AdamW 优化器进行参数更新。
- 学习率调度**：采用余弦退火学习率调度器，逐渐降低学习率以促进模型收敛。

3.1 ViT 核心理念

- 将输入图像切分为固定大小的 patch，并映射为 token 序列。
- 在 token 序列前拼接可学习的 [CLS] token，用于全局分类表征。
- 叠加位置编码后送入 Transformer Encoder，最终通过分类头输出类别概率。

3.2 本实验模型结构

- Patch 大小: 4 x 4
- Transformer 宽度: dim=256
- 层数: depth=6
- 注意力头数: heads=8
- MLP 隐藏维度: mlp_dim=512
- Dropout: 0.2

3.3 核心代码实现

Patch Embedding:

```
class PatchEmbedding(nn.Module):
    def __init__(self, channels, patch_height, patch_width, dim):
        super().__init__()
        self.patch_height = patch_height
        self.patch_width = patch_width
        patch_dim = channels * patch_height * patch_width
        self.norm1 = nn.LayerNorm(patch_dim)
        self.proj = nn.Linear(patch_dim, dim)
        self.norm2 = nn.LayerNorm(dim)

    def forward(self, x):
        b, c, h, w = x.shape
        p1, p2 = self.patch_height, self.patch_width
        x = x.reshape(b, c, h // p1, p1, w // p2, p2)
        x = x.permute(0, 2, 4, 3, 5, 1).reshape(b, (h // p1) * (w // p2), p1 * p2
        * c)
        x = self.norm1(x)
        x = self.proj(x)
        x = self.norm2(x)
        return x
```

训练与验证主循环 (简化):

```
for epoch in range(1, args.epochs + 1):
    train_loss, train_acc = train_one_epoch(
        epoch, net, trainloader, optimizer, criterion, device
    )
    val_loss, val_acc, best_acc =
    evaluate(epoch, net, testloader, optimizer, criterion, scheduler, args, best_acc, device,)
    if args.cos:
        scheduler.step()

    writer.writerow(
        [
            epoch,
            f"{train_loss:.6f}",
            f"{train_acc:.6f}",
            f"{val_loss:.6f}",
            f"{val_acc:.6f}",
```

```

    ]
)
f.flush()

print(
    f"Epoch {epoch}/{args.epochs} | train_loss: {train_loss:.4f} | "
    f"train_acc: {train_acc:.2f}% | val_loss: {val_loss:.4f} | val_acc: "
    f"{val_acc:.2f}%"
)

```

模型配置参数

```

OPTIMIZER = "adamw"
LOSS_FN = "ce"
WEIGHT_DECAY = 0.05
MOMENTUM = 0.9
LABEL_SMOOTHING = 0.0
BATCH_SIZE = 32
IMAGE_SIZE = 64
USE_COSINE = False

```

自注意力机制

```

class Attention(nn.Module):
    def __init__(self, dim, heads=8, dim_head=64, dropout=0.0):
        super().__init__()
        inner_dim = dim_head * heads
        project_out = not (heads == 1 and dim_head == dim)

        self.heads = heads
        self.dim_head = dim_head
        self.scale = dim_head ** -0.5

        self.norm = nn.LayerNorm(dim)
        self.attend = nn.Softmax(dim=-1)
        self.dropout = nn.Dropout(dropout)
        self.to_qkv = nn.Linear(dim, inner_dim * 3, bias=False)

        self.to_out = (
            nn.Sequential(
                nn.Linear(inner_dim, dim),
                nn.Dropout(dropout),
            )
            if project_out
            else nn.Identity()
        )

    def forward(self, x):
        x = self.norm(x)

```

```
qkv = self.to_qkv(x).chunk(3, dim=-1)
q, k, v = [t.reshape(t.shape[0], t.shape[1], self.heads,
self.dim_head).permute(0, 2, 1, 3) for t in qkv]

dots = torch.matmul(q, k.transpose(-1, -2)) * self.scale
attn = self.dropout(self.attend(dots))

out = torch.matmul(attn, v)
out = out.permute(0, 2, 1, 3).reshape(out.shape[0], out.shape[2],
self.heads * self.dim_head)
return self.to_out(out)
```

4. 训练配置与流程

4.1 数据处理

- 训练集增强: `RandomResizedCrop + RandomHorizontalFlip + Normalize`
- 验证集处理: `Resize + CenterCrop + Normalize`

4.2 训练配置

- 优化器: `AdamW`
- 损失函数: `CrossEntropyLoss`
- 学习率: `1e-3`
- 权重衰减: `0.05`
- Batch Size: `32`
- 计划 Epoch: `100`

5. 实验结果

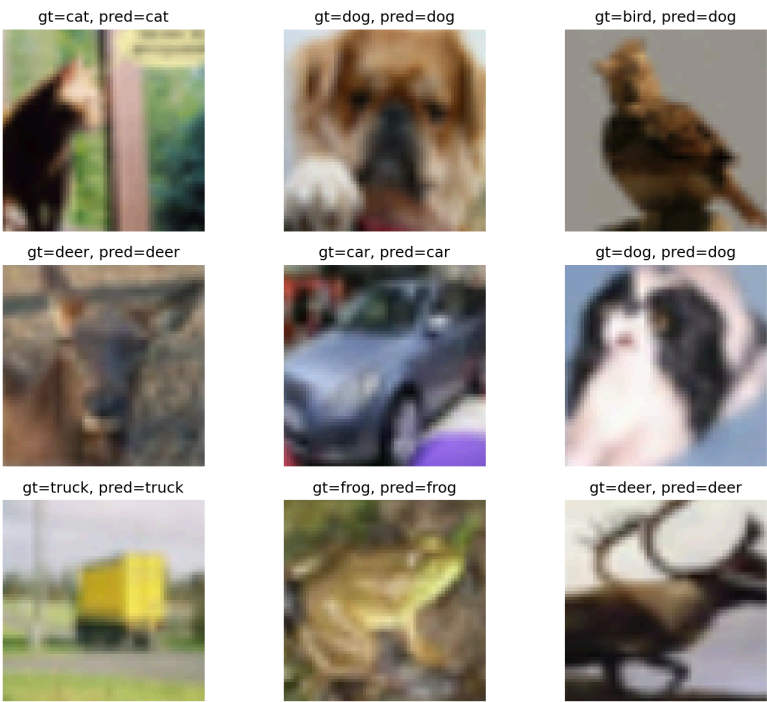
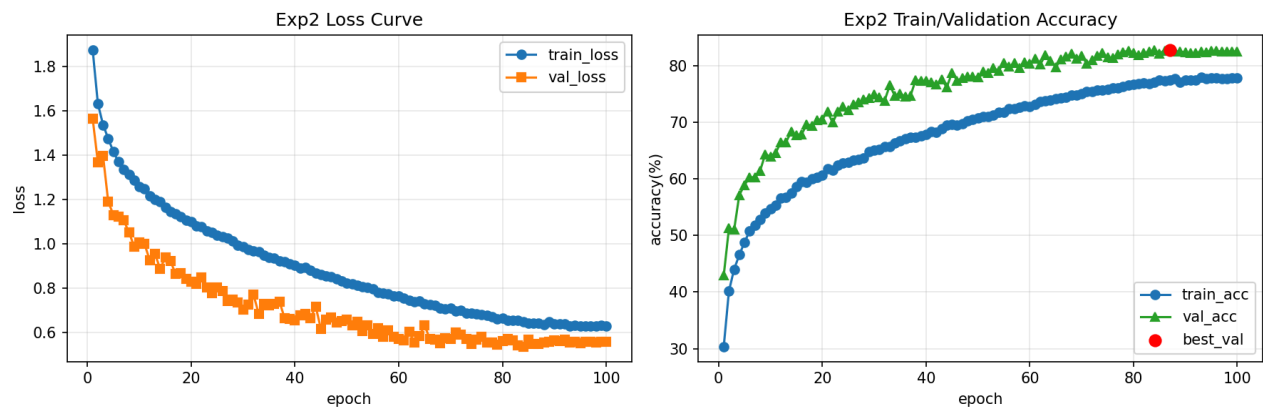
5.1 定量结果 (来自 `log/history.csv`)

- 第 1 轮: `val_acc = 42.94%`, `val_loss = 1.564983`
- 最优轮次: 第 87 轮, `val_acc = 82.81%`, `val_loss = 0.549812`
- 最后一轮 (第 100 轮): `val_acc = 82.53%`, `val_loss = 0.557326`

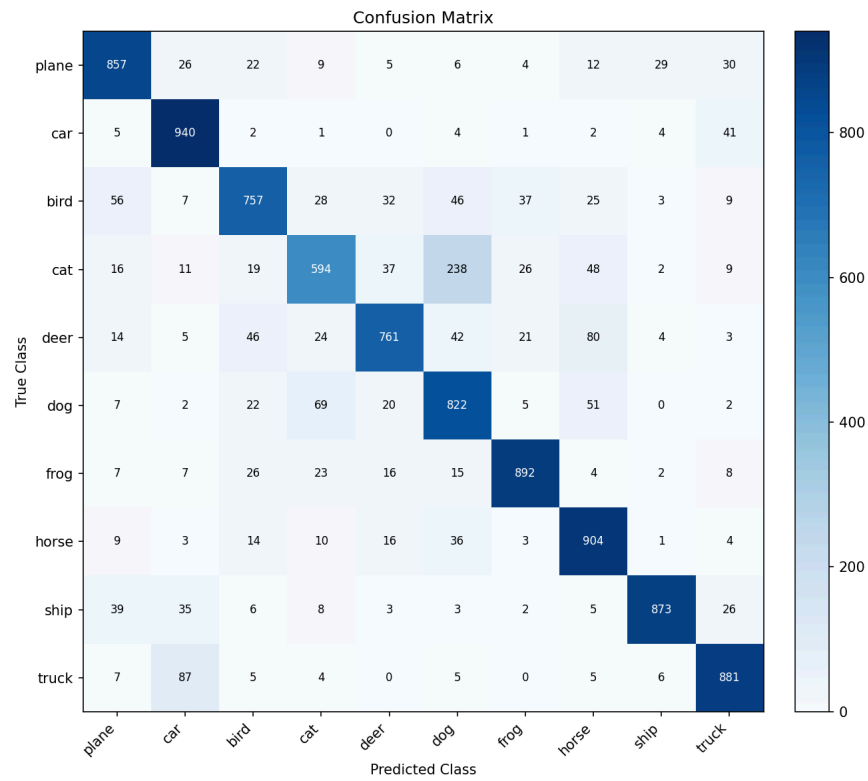
可见模型在训练过程中稳定收敛, 验证准确率从 42.94% 提升到 82.81%, 总提升约 39.87 个百分点。

5.2 可视化结果

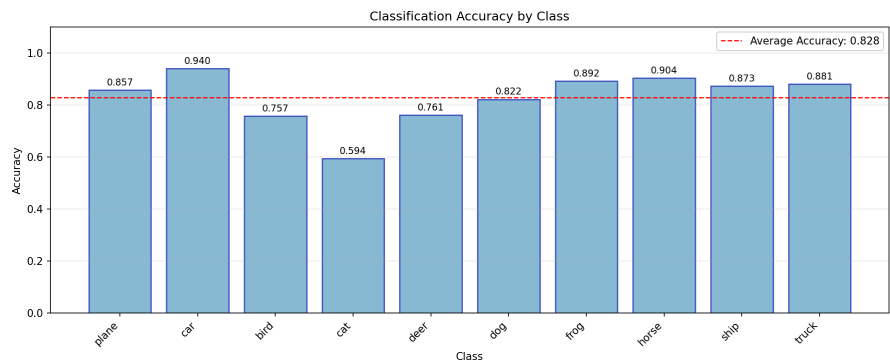
• 训练曲线:



• 测试集预测样例:



- 分类判断热力图：



- 分类统计柱状图：

6. 实验总结

- 本实验完成了 ViT 在 CIFAR-10 上的端到端训练与评估流程，模型达到了 **82.81%** 的最高验证准确率。
- 从曲线看，训练与验证指标整体趋势一致，未出现明显发散，说明当前优化设置具备较好的稳定性。
- 后续可从数据增强强度、学习率调度策略（如余弦退火）和更长训练轮数等方面继续优化准确率。